Bottleneck — specification v0.2

Nov 11, 2026 · @Damjan

What changed in v0.2 (after five independent reviews: simulation, frontend, backend/security, domain, implementer):

- Phases reordered: a deployed demo on a built-in venue comes first (about 2 weeks); the editor comes after the backend.
- The .NET backend gets a real job: a server-side batch runner for multi-seed runs and "how many lanes?" sweeps.
- Validation tests rewritten so they are mathematically sound (M/D/1, Little's law); determinism scoped and enforced.
- One agent state enum everywhere; queue spillback and car–pedestrian crossings added; security screening is its own stage.
- Headline metrics depend on event type; arrival presets; corrected default timings; scan failures and plate misreads.
- Replay/scrubber model, worker data path and performance targets defined; Next.js downgraded to Expected and client-only.
- Sharing keys, AI cost limits, uploads, schema mechanism and deployment hardened.
- Every acceptance check is tagged **[test]** (an automated test proves it) or **[human]** (a person checks it).

1. What Bottleneck is

Bottleneck is a browser-based simulator that shows where a venue's or parking lot's entry and exit will jam before the event happens. You draw the site over a map screenshot, place pedestrian gates, vehicle barriers and parking, enter how many people and cars you expect, and watch the simulation run.

The core loop:

1. Draw the site.
2. Set a scenario.
3. Press Play and watch queues form.
4. Change something: add a lane, break a gate, swap the barrier type.
5. Compare the results.

The key questions it answers: how many lanes are needed to keep the wait under a target, and when must the gates open? For events with a fixed start (concert, match): how many people are still outside at the start? For open-door events (a fair): what is the 95th-percentile wait at peak? For parking: how far does the entry queue back up, and how long until the lot is empty after the event?

Who it is for:

- **Primary: portfolio.** A deployed, well-tested, visually impressive project for junior .NET / full-stack applications in Serbia. It shows simulation, real-time rendering, a typed domain model, a .NET backend with a background job runner, and AI integration.
- **Imagined user (shapes the design):** a planner at an access-control company or venue, such as one installing turnstiles and barriers at the Belgrade Fair, deciding how many lanes to install and where.

Success looks like: a stranger opens the live demo, loads the Belgrade Fair example, presses Play, and within two minutes sees a jam form at a gate. They then fix it by adding a lane and see the before/after numbers side by side.

Prior art. Professional tools such as PTV Vissim and Oasys MassMotion, and consultancies like SimPlan, do this for engineers. Bottleneck is a lightweight web version focused on gates, barriers and parking. It is not a scientific crowd-physics tool.

2. How to read this spec

Every requirement has one of three levels. When this spec is silent, the implementer decides, writes the decision down, and keeps going.

Level	Meaning
Fixed	Non-negotiable. Changing it needs the product owner's approval. All Fixed rules are also listed together in section 7.
Expected	The default. Deviate only with a good reason, and record that reason.
Open	The implementer chooses libraries, structure, naming and UI details. Pick what is idiomatic and maintainable.

Decision records. Write one for any decision that changes the data model, the simulation rules, the API contract, the auth or storage approach, or adds a runtime dependency. Store them in `docs/decisions/NNN-title.md` with four short parts: context, decision, alternatives considered, consequences. Skip records for routine choices such as naming or folder layout.

Product questions. Every product question this spec raises has a default in section 10. Use the default, flag it in the phase summary, and keep going. Stop and ask only when a choice would contradict a Fixed rule or make a later phase impossible.

3. Core concepts

A venue is a scaled drawing made of typed elements. A scenario is a set of inputs run against a venue. A run is one seeded execution of a scenario, and it produces metrics. The exact fields and the storage shape are Open, as long as these concepts and their rules exist.

Concept	What it is	Rules
Venue	One site: background image, scale, and all drawn elements	Has a name and a scale. All geometry is stored in metres, never pixels.
Background	The map image the venue is drawn on	Display only; the simulation never reads its pixels. Replaceable without breaking elements.
Walkable area	Polygons where pedestrians may walk	Everything else blocks pedestrians. Obstacles are holes.
Road	Polylines vehicles drive along, with direction and lane count	Vehicles move only on roads and parking aisles. Each road segment has a storage length (how many cars fit).
Crossing	A place where a walkway crosses a road or aisle	Cars yield to pedestrians on the crossing; pedestrians wait for a gap when cars are queued across it.
Source	Where agents appear: bus/tram stop, street entrance, road inflow	Has a type (pedestrian or vehicle) and a share of arrivals.

Concept	What it is	Rules
Screening stage	Security or bag-check lanes in front of the gates	Its own lanes, staffing and service time. Optional per gate group.
Pedestrian gate	A group of identical lanes (turnstiles) between outside and inside	Lane count, service-time distribution, scan-failure rate, state (open or failed) per lane.
Vehicle barrier	A group of entry or exit lanes on a road	Lane count, service type (ticket, plate recognition, card), service time, misread rate.
Parking lot	A zone with stalls and aisles, linked to roads by its barriers	Capacity = its stalls. Has a walking exit and an optional pay station.
Stall	One parking space	Free or occupied; position and orientation.
Pay station	A machine where pedestrians pay before returning to their car	A pedestrian queue with its own service time.
Destination	Where pedestrians go once inside (hall, stage)	Pedestrians finish when they reach it.
Scenario	All inputs for one run	Event type and preset, attendance, car share, gate opening time, event start/end, curves, overrides, timed events, seed.
Timed event	Something that happens mid-run	Lane fails or reopens, bus burst (N people at once), lot closes.
Agent state	Where an agent is in its journey	Exactly one of: Pending, Walking, Queuing, InService, Inside, Driving, Parked, Departed, TurnedAway, Diverted. Used everywhere (engine, metrics, tests).
Run	One execution of a scenario	Seeded and deterministic (section 7.1). Stores metrics plus the parameter set used, not every frame.

Concept	What it is	Rules
Job	A server-side batch of runs	Multi-seed batch or lane sweep, executed by the backend (section 8).
Comparison	Two or more runs shown side by side	Runs must share a venue, or the UI says that they don't.

4. Simulation model

Bottleneck combines two layers: a time-stepped layer that moves people and cars a little each tick, and an event-driven queue core where every lane (screening, gate, barrier, pay station) serves one agent at a time with exact service times. It aims to be plausible, testable and explainable, not physically exact.

In plain words: dots walk towards the gates, line up, wait while the one in front scans a ticket, then walk in. Cars do the same on roads. Cars that park turn into the people who were inside them, and later those people walk back to their cars.

4.1 Engine (Fixed)

- Pure TypeScript with no DOM access. It runs in a Web Worker in the browser and headless in Node (tests, and the server-side batch runner in section 8).
- **Movement step:** fixed 0.25 s of simulated time. Playback speed (1× to 60×) changes how many steps run per real second, never the step size.
- **Queue core:** service starts and completions are scheduled as events with exact (unrounded) times, so a 3.1 s service takes 3.1 s, not 3.25 s. Each tick processes, in this fixed order: due arrivals, due service completions, lane assignments, movement, density update, metric sampling.
- **Determinism rules:** one documented seeded PRNG (sfc32 or mulberry32 using `Math.imul`); engine arithmetic uses only + − × ÷ and `Math.sqrt`; any other math function (exp, sin, pow, log) comes from an in-repo implementation or lookup table; iteration is always ordered by agent id or element id, never by Map/Set insertion or hash order.
- **Snapshots to the renderer:** about 20 per second of wall-clock time, as a structure-of-arrays `Float32Array` (x, y, state, heading per agent) sent as a Transferable and double-buffered. No SharedArrayBuffer. If the renderer falls behind, snapshots are

dropped, never queued.

4.2 Pedestrians (Expected)

- **Navigation:** the walkable area is rasterised into a 0.5 m grid. Each target (screening, gate, destination, lot exit, pay station, own car) gets a precomputed flow field.
- **Density:** measured on a 2 m grid with Gaussian smoothing ($\sigma \approx 0.7$ m), updated every tick.
- **Speed:** each agent's free speed is drawn from a truncated normal (section 5). Actual speed follows the Weidmann speed–density relation, $v = v_0 \cdot (1 - \exp(-\gamma \cdot (1/\rho - 1/\rho_{max})))$ with $\gamma \approx 1.913$ and $\rho_{max} \approx 5.4$ people/m². Agents already in a queue line are not moved by this curve.
- **Lane choice:** when within a decision distance of a gate group, an agent picks a lane with a logit (probabilistic) choice over walk time + expected wait. The expected wait counts agents already committed to that lane, not only those standing in it, which prevents everyone piling into the same lane in one tick. A configurable share simply picks the nearest lane.
- **Jockeying:** at most once, only to a lane with at least 3 fewer people, and to the back of that queue. When a lane fails, its queue is reassigned with the same rule, preserving arrival order.
- **Queue storage and spillback:** each lane has a queue area of fixed length. When it is full, further arrivals wait in the walkable area in front of it and count as queuing, so jams visibly spill back.

4.3 Vehicles (Expected)

- Cars move along the road graph with simple car-following (minimum gap, no overtaking within a lane) and a free speed per road.
- **Spillback:** a car cannot enter a road segment that is full. Barrier queues therefore back up along the road, and the engine records when they reach a marked public-road boundary.
- **Crossings:** cars stop if a pedestrian is on a crossing; pedestrians wait at a crossing when cars are stopped across it.
- **Barriers** serve one car per lane at a time. A plate-recognition misread sends the car through a slower fallback (ticket or intercom) with the misread probability from section 5.

- **Parking:** a car picks a free stall by a configurable rule (nearest to entry or nearest to walking exit) and drives there along the aisles. If the lot is full, the overflow rule applies: queue at the entry, divert to another lot, or leave (TurnedAway).
- **Cars become people:** after parking, the occupants (count drawn from the occupancy distribution) leave the car after a fixed exit delay and walk from the stall to the lot's walking exit. They remember their car.

4.4 Departures (Expected)

- After the event ends, agents leave following the departure curve. Car users walk back to their own car; the car then drives to an exit barrier.
- Payment per lot: either at a pay station on foot (a pedestrian queue before walking to the car), or at the exit barrier (longer service time).
- The headline departure result is the time until the lot is empty, plus the longest exit-queue length.

4.5 Metrics (Fixed)

- **Per lane** (screening, gate, barrier, pay station): queue length over time, waiting time Wq (from joining the queue to service start; mean, p95, max), throughput per minute, utilisation. Waits and utilisation are reported over the window from the lane's first arrival to its last service, and separately before and after the event start. A lane failure splits its utilisation into open and failed periods.
- **Venue-wide, every event type:** p95 wait at the peak arrival hour, share of attendees inside by a chosen time T (default: 10 min before the start), latest gate opening time that still meets the target (from a sweep).
- **Fixed-start events only:** people still outside at the start = agents whose gate service starts after the event start.
- **Parking:** occupancy over time, average stall-search time, cars turned away or diverted, maximum entry-queue length in metres and whether it reached the public road, time until the lot is empty.
- **Heatmap:** the density grid is streamed live; a time-aggregated grid is saved with the run. A per-moment heatmap exists only live or in replay.

5. Default parameters

No real data is available, so every number below is an approximate assumption from general knowledge, not a measurement. Each one must be editable per venue or scenario, stored with every run, and labelled "assumed" in the UI (Fixed). Replace them if real data arrives.

Parameter	Default (assumed)	Scope
Pedestrian free walking speed	Normal, mean 1.3 m/s, SD 0.25 m/s, truncated to 0.5–2.2 m/s	Scenario
Speed–density relation	Weidmann (section 4.2), $\gamma \approx 1.913$, $\rho_{max} \approx 5.4$ people/m²	Venue
Security screening	8–20 s per person, mean 12 s (shifted lognormal); 1 staffed lane per 2 gate lanes	Screening stage
Turnstile scan (QR or card)	2.5–7 s per person, mean 4 s (shifted lognormal)	Gate
Scan failure	3% of scans fail; retry adds 5–15 s; a second failure sends the person to a manual lane	Gate
Share picking the nearest lane only	30%	Scenario
Car barrier entry, ticket dispensed	5–10 s per car	Barrier
Car barrier entry, plate recognition	4–8 s per car, including the arm cycle	Barrier
Plate misread	5% of cars; fallback adds 15–30 s	Barrier
Car barrier exit, prepaid or plate recognition	4–8 s per car	Barrier
Car barrier exit, pay at barrier	20–40 s per car	Barrier
Pay station	30–60 s per person	Lot
People per car	1: 35%, 2: 35%, 3: 18%, 4: 12% (mean ≈ 2.1)	Scenario

Parameter	Default (assumed)	Scope
Share of visitors arriving by car	40%	Scenario
Road speed on site / in parking aisles	25 km/h / 8 km/h	Venue
Time to get out of a parked car	20 s, then walk from stall to the lot exit	Lot
Stall size (90°) / two-way aisle width	2.5 × 5.0 m / 6.0 m	Lot

Arrival presets (editable as a few points the UI smooths into a curve):

Preset	Arrival shape (assumed)	Departure shape (assumed)
Concert	Gates open 90 min before; peak 45–60 min before; 10% arrive in the first 30 min after start	80% leave within 30 min of the end
Stadium	Gates open 90 min before; peak 30–45 min before; 5% arrive after kick-off	85% leave within 20 min of the final whistle
Fair / open-door	Flat with a late-morning and an early-afternoon bump over opening hours; no fixed start	Spread over the last 3 hours
Conference	Peak 30–15 min before the first session	Spread over the last hour

Synthetic scan logs (Expected). A generator turns a run into fake turnstile and barrier logs (timestamp, lane, event type). An importer reads logs in that format and fits the arrival curve and service times. Service time is estimated from busy-period inter-departure intervals only: the first departure after an idle period is excluded, because it would include idle time.

6. Feature areas

Eight feature areas, each mapped to a phase in section 9. Nothing in the core project calls a paid API: the only paid feature, the AI draft (6.8), is optional and off by default.

6.1 Example venues (built in)

Must:

- Ship pre-drawn venues as versioned JSON files in the repo, loadable with one click and usable before any editor or backend exists:
  - **Belgrade Fair, fair day:** open-door, several pedestrian entrances, car parks with barriers.
  - **Belgrade Fair, concert in a hall:** fixed start, screening + turnstiles, exit rush.
  - **Stand-alone parking lot:** two entries, one exit, pay station — the exit-rush demo.
- Each example comes with 2–3 ready scenarios (e.g. "3 lanes vs. 5 lanes", "gate B fails at 19:30").
- The examples are drawn by hand by the coding agent from an OpenStreetMap layout of the area (with attribution); exact real-world accuracy is not required, plausibility is.

6.2 Simulation view

Must:

- Play, pause, step, restart; speed 1× to 60×.
- **Replay and scrubbing, two modes:** while a run plays, the worker keeps keyframes (1 per simulated second, positions quantised to Int16 decimetres, capped at about 200 MB); the scrubber seeks within them. For a run loaded later, or past the cap, playback re-runs the seeded simulation to the chosen time, which determinism makes exact. Charts are always scrubbable.
- The renderer interpolates between the last two snapshots at speeds below 10×; at 10× and above it draws the latest snapshot as is.
- Pedestrians and cars drawn over the venue; queues visible as lines; failed lanes marked by both colour and shape. Heatmap overlay toggle.
- Live side panel: clock relative to event start, agents per state, queue length per lane, parking occupancy. Click any lane or lot for its charts.
- A `?perf=1` overlay showing frame time (p50/p95), worker tick time and snapshot bytes per second.

Open: visual style, sprites, chart library (uPlot or ECharts suggested for dense time series). Respect `prefers-reduced-motion`.

6.3 Scenarios and comparison

Must:

- Create, duplicate and edit scenarios: event type and arrival preset, attendance, car share, gate opening time, event start and end, overrides, seed.
- Timed events on a scenario timeline: lane fails or reopens at T, bus burst of N people at source S at T, lot closes at T.
- Comparison: headline metrics side by side, overlaid queue charts per lane, highlighted differences ("p95 wait at peak: 14 min → 4 min"). Compared runs use the same seeds (common random numbers), so differences come from the change, not from luck.

6.4 Batch runs and lane sweep (server-side jobs)

Must:

- **Batch:** run one scenario over N seeds (default 20) and show the mean with a 95% confidence interval for each headline metric.
- **Lane sweep:** for one gate group or barrier, run lane counts 1…k automatically and plot p95 wait against lanes. Mark the smallest count that meets a user-set target wait. The same mechanism sweeps the gate opening time.
- Both run as backend jobs (section 8): submitted, queued, cancellable, with progress shown. Results are computed by the server, so they are trusted, unlike client-reported runs.
- Before the backend exists (Phases 1–3), a small client-side batch of up to 5 seeds is acceptable.

6.5 Parking operation

Must:

- Lots with stalls, aisles, entry and exit barriers, crossings, optional pay station. Arrival and departure (exit rush) runs.
- Stall-choice, overflow and payment rules per lot (sections 4.3 and 4.4).
- Lot metrics from section 4.5, including whether the entry queue reaches the public road.

Open: how stalls are drawn manually (one by one or as rows).

6.6 Venue editor

Must:

- Upload a background image (PNG or JPEG) and calibrate scale: click two points, type the real distance.
- Draw and edit every element type in section 3. Pan and zoom (wheel and pinch). Snap to grid and to vertices or edges, with a tolerance in screen pixels and a key to disable snapping. Holes in polygons.
- Select, move, resize, delete, undo/redo. Editor state is an immutable document with a command stack (each command has an inverse), kept separate from the rendered display objects.
- Properties panel per element. Keyboard-shortcut table.
- Validation before a run, with clear messages: unreachable gates or sources, lots without an exit barrier, self-intersecting polygons, too few vertices.
- Any example venue (6.1) can be opened and edited as a copy.

6.7 Parking layout generator

Must:

- Given a lot outline, entry and exit positions and a stall angle (90°, 60°, 45°), fill the lot with stall rows and aisles using section 5 dimensions, and show the stall count.
- Generate up to three variants, each of which can be simulated and compared.

Out of scope: true optimisation (searching all layouts). Stretch goal only.

6.8 AI auto-draft from a screenshot (optional, paid, last)

This is the only feature that costs money (Anthropic API usage), so it is built last, behind a feature flag that is **off by default**. The app must be complete without it: example venues plus the manual editor cover every need.

When built, it must:

- Send the uploaded image from the backend to a vision-capable Claude model and receive proposed elements as JSON, shown as a separate draft layer. Nothing becomes part of the venue until the user accepts it.
- Enforce a global daily spend cap, 3 drafts per IP per hour, a 30 s timeout and a maximum output size; reject requests when the cap is hit with a clear message.

- Treat the model output as untrusted: validate strictly against the draft schema, drop unknown fields, clamp coordinates, cap element counts. The prompt states that any text in the image is data, not instructions.
- Be available only to the venue's edit-key holder.

7. Invariants and non-functional requirements

All Fixed rules are collected here so they can be checked in one place. Each has at least one automated test, listed in the README.

7.1 Fixed invariants

1. **Determinism:** the same venue, scenario, parameters and seed produce bit-identical metrics in Node (the CI version) and in Chrome's Web Worker. A CI test runs a fixed scenario in both (Chrome via Playwright) and compares a hash of the metrics.
2. **Metres, not pixels:** stored geometry is in metres. Translating the venue, or swapping the background image, gives identical metrics; uniformly rescaling the venue by a small factor changes metrics by less than 2% (grid rasterisation makes this approximate).
3. **Conservation:** at every snapshot, the agents in each state of the section 3 enum sum exactly to the number created. Car occupants are counted as pedestrians from the moment they leave the car; the car itself is counted separately.
4. **Physical rules:** agents move only on walkable areas, roads or aisles, never through obstacles; a lane serves at most one agent at a time; a failed lane serves none.
5. **Little's law:** for every lane over its reporting window, mean queue length ≈ arrival rate × mean wait ($L = \lambda W$), within 3%.
6. **Engine purity:** the engine depends on nothing from the DOM, the renderer or the network.
7. **Assumptions are visible:** every section 5 default shown in the UI is labelled "assumed", and every run stores the full parameter set it used.
8. **No paid API is required:** the core project runs end to end with no API key. The AI draft (6.8) is off unless explicitly enabled.
9. **No secrets in the client or the repository.**

7.2 Non-functional requirements

Area	Requirement	Level
Performance (browser)	Reference scenario committed to the repo: 10,000 pedestrians + 2,000 cars at 8× speed. Measured with the `?perf=1` overlay over 60 s: p95 frame time under 33 ms. The machine used is recorded in the README.	Expected
Performance (headless)	`npm run bench` runs the same scenario for 4 simulated hours; reported in CI as information. Target: under 60 s.	Expected
Responsiveness	The UI never freezes during a run; the engine stays off the main thread.	Fixed
Quality	Unit tests for the engine and the API; CI runs them on every push. Playwright smoke tests for loading an example, running it, and (once built) drawing, undoing and saving in the editor.	Expected
Validation tests	The queueing test family in section 9 (Phase 1) plus Little's law on every run in CI.	Fixed
Device	Desktop-first. On a phone, shared runs show headline metrics, comparisons and charts; animated replay is optional. Device pixel ratio capped at 2.	Expected
Accessibility	Colour-blind-safe palette; states shown by shape as well as colour; key metrics as numbers; `prefers-reduced-motion` respected.	Expected
Language	English UI; strings kept out of components.	Expected
Security	Section 8 rules for keys, uploads, rate limits and headers. A short `docs/security.md` threat model.	Fixed
Operability	One command runs the full stack locally with the example venues loaded. Structured logs and a health endpoint on the backend.	Expected

8. Technology and architecture

Interactive simulation runs in the browser for smooth animation. The .NET backend stores venues, scenarios and results, and runs heavy work as background jobs: multi-seed batches and lane sweeps. Until Phase 4 the app is a static site with no backend.

Layer	Choice	Level
Frontend	Next.js + TypeScript + Tailwind, client components only (`'use client'`, no SSR for the editor or simulation), static export where possible. Vite + React is acceptable with a decision record.	Expected
Rendering	PixiJS (WebGL), particle container or instanced sprites; render-on-demand while idle in the editor	Expected
Simulation engine	Pure TypeScript package; runs in a Web Worker and in Node	Fixed
Backend	ASP.NET Core Web API on .NET 10	Fixed
Database	PostgreSQL via EF Core; venue geometry and scenarios as `jsonb`, with id, name, schema version and timestamps as columns	Fixed
Job runner	.NET `BackgroundService` with a `Channel<T>` queue, executing the same engine headless in a Node sidecar container	Expected
Tests	Vitest (engine, frontend), xUnit + WebApplicationFactory + Testcontainers (API), Playwright (browser)	Expected
AI	Anthropic API, optional, Phase 8 only, behind a flag that is off by default	Expected
Hosting	Phases 1–3: free static hosting (Vercel, Cloudflare Pages or GitHub Pages). Phase 4 on: one small VPS (about €5/month, or free with student credits)	Expected

How the parts fit:

1. The browser holds the example venues, the editor, the renderer and the engine worker. The engine never touches the network.

2. The frontend saves venues, scenarios and runs through the API.
3. Jobs: the frontend submits a batch or sweep; the API stores it, queues it, and the job runner executes the runs through the Node engine sidecar, reporting progress. Jobs can be cancelled, are idempotent by request id, and survive an API restart (queued jobs are reloaded from the database).
4. Results computed by jobs are trusted. Runs uploaded from the browser are labelled `clientReported`, schema-validated, capped at 256 KB, and limited to 50 per venue.

The backend's engineering checklist (Expected): layered or vertical-slice structure; `ProblemDetails` errors; OpenAPI with an interactive UI; structured logging (Serilog); health checks; optimistic concurrency on venue updates so two tabs cannot silently overwrite each other; rate limiting on all writes; retention cleanup of abandoned demo data.

Sharing without logins (Fixed):

- A venue has a public `id`, a regenerable `readToken`, and an `editKeyHash`.
- The edit key is 256 random bits, shown once, stored only as a SHA-256 hash, compared in constant time, and sent only in an `X-Edit-Key` header.
- The private edit link is `/v/{id}#k={key}`. The fragment never reaches the server, and the app removes it from the address bar after reading it.
- `Referrer-Policy: no-referrer`; request headers and bodies are never logged on write routes. Endpoints exist to rotate the key and to delete the venue.
- The check is an ASP.NET authorization policy, so real accounts can replace it later without touching endpoints.

Uploads (Fixed): PNG or JPEG only, checked by magic bytes; at most 5 MB and 4096 px per side; decoded and re-encoded with ImageSharp, which strips metadata; stored under server-generated names outside the web root; served with a fixed content type, `nosniff` and a CSP; per-venue storage quota.

Shared schema (Fixed): the venue, scenario and run formats are written once as JSON Schema in `/schema`. TypeScript types and C# classes are generated from it, and CI fails if the generated code is out of date. The backend validates every write against the same schema. Shared fixture files are loaded by both test suites. Every stored document carries `schemaVersion`; the backend migrates old documents on read through an ordered chain of tested migration functions, and rejects versions newer than it knows.

Deployment (Expected, from Phase 4): Docker Compose on the VPS behind Caddy (automatic HTTPS); containers run as non-root; Postgres not exposed publicly; secrets in an untracked `.env` with restricted permissions; nightly `pg_dump` plus upload copies to off-box storage, with one restore tested and documented; CORS limited to the frontend's origin.

9. Delivery phases

Build in order; every phase ends with something that runs and is deployed. The order puts a working, public demo first, so applications can start early.

Milestones: a demo worth putting on a CV after Phase 2 (about 3 weeks); the full-stack .NET story after Phase 4 (about 6 weeks). Phases 7 and 8 are optional. Durations assume the coding agent builds and the owner reviews part-time.

[test] = an automated test proves it, run in CI. **[human]** = the owner checks it by looking.

Phase 0: Foundation (1–2 days)

Scope: monorepo (engine package, web app; the API is added in Phase 4); JSON Schema with TypeScript generation; seeded PRNG and tick loop; Vitest; CI; static deployment pipeline.

- [ ] **[test]** Two runs with the same seed give identical output.
- [ ] **[test]** The same tiny scenario gives the same metrics hash in Node and in Chrome's Web Worker (Playwright).
- [ ] **[human]** An empty page is live on free static hosting.

Phase 1: Pedestrian simulation on a built-in venue (1.5–2 weeks)

Scope: sections 4.1, 4.2, 4.5 for pedestrians; screening stage, gates, scan failures; simulation view with replay and heatmap (6.2); the two Belgrade Fair example venues (pedestrian parts) with arrival presets; timed lane failure; perf overlay.

- [ ] **[test]** Deterministic queue (D/D/1): at 80% utilisation the queue stays 0; at 125% the queue at time t equals $(\lambda - \mu)\cdot t \pm 1$.
- [ ] **[test]** M/D/1 at 70% utilisation, Poisson arrivals injected directly at one lane, 20 seeds × 10 h after a 1 h warm-up: mean Lq is within 5% (or the 95% CI half-width, whichever is larger) of the theoretical 0.817.

- [ ] **[test]** M/G/1 with the default lognormal scan time at 80% utilisation: mean Wq matches the Pollaczek–Khinchine formula by the same rule.
- [ ] **[test]** Little's law and conservation hold on every lane and snapshot of the example runs.
- [ ] **[test]** Failing 2 of 4 lanes 30 min before the start raises the remaining lanes' maximum queue by at least 30% compared with the same seed without the failure.
- [ ] **[human]** The performance target in 7.2 is met on the reference machine.
- [ ] **[human]** The live demo loads the concert example and a jam forms within 2 minutes of pressing Play.

Phase 2: Scenarios and comparison (about 1 week)

Scope: section 6.3; client-side batch of up to 5 seeds; synthetic scan-log generator and importer. Everything still client-side.

- [ ] **[test]** Comparing 3 vs. 5 lanes on the concert example shows highlighted differences (Playwright).
- [ ] **[test]** A generated scan log, re-imported, recovers the mean scan time within 10%.
- [ ] **[human]** README with a GIF, the live link and the how-it-works notes. **First CV milestone.**

Phase 3: Vehicles and parking (about 1.5 weeks)

Scope: sections 4.3, 4.4, 6.5 — roads with spillback, crossings, barriers with misreads, lots, stalls, cars turning into people, pay stations, departures; the parking-lot example venue.

- [ ] **[test]** A parked car produces exactly its occupant count as pedestrians.
- [ ] **[test]** Over 10 paired seeds, plate recognition gives a lower mean entry queue than ticket entry, and the 95% CI of the paired difference excludes 0.
- [ ] **[test]** Departure with N cars and one exit lane as the bottleneck (service time s): time to empty is at least N·s and within 10% of N·s plus walking and driving time.
- [ ] **[test]** A full lot applies its overflow rule; turned-away cars are counted; conservation holds.
- [ ] **[test]** An entry queue reaching the public-road boundary is recorded with its time.

Phase 4: .NET backend and jobs (about 1.5 weeks)

Scope: section 8 in full — API, PostgreSQL, schema validation and migrations, edit-key sharing, run storage, job runner, batch and lane sweep (6.4), security rules, VPS deployment with Caddy and backups.

- [ ] **[test]** Integration tests (Testcontainers): a missing or wrong edit key is rejected; rotating the key invalidates the old one; concurrent updates are detected.
- [ ] **[test]** A 20-seed batch returns mean and 95% CI; a job can be cancelled; a queued job survives an API restart.
- [ ] **[test]** A lane sweep on the concert example returns the smallest lane count that meets the target wait.
- [ ] **[test]** A job run and a browser run with the same seed produce the same metrics hash.
- [ ] **[test]** A wrong file type or oversized image is rejected; exceeding a rate limit returns 429.
- [ ] **[human]** The app is live on the VPS over HTTPS; one backup restore has been tested. **Second CV milestone.**

Phase 5: Venue editor (1.5–2 weeks)

Scope: section 6.6.

- [ ] **[test]** After two-click calibration, a 100 m distance measures 100 m ± 1%.
- [ ] **[test]** Draw a venue (walkable area, source, gate group with 4 lanes, destination), save, reload: unchanged (Playwright).
- [ ] **[test]** Undo/redo returns the document to exactly the previous state.
- [ ] **[test]** Validation flags an unreachable source and a self-intersecting polygon.
- [ ] **[test]** A read-only link shows the venue but rejects edits.

Phase 6: Showcase (3–5 days)

Scope: polish, an onboarding tour of the examples, a performance pass, final README (architecture, decisions, how to run, limitations), `docs/security.md`, a short demo video.

- [ ] **[human]** A first-time visitor loads an example, runs it and compares two scenarios without instructions.
- [ ] **[test]** Every invariant in 7.1 has a passing test, listed in the README.

Phase 7 (optional): Parking layout generator

Scope: section 6.7.

- [ ] **[test]** A 50 × 100 m rectangle at 90° gives the stall count of a hand calculation from the default dimensions.
- [ ] **[test]** Three variants can each be simulated and compared.

Phase 8 (optional, paid): AI auto-draft

Scope: section 6.8. Only if the owner decides to pay for API usage.

- [ ] **[test]** With the flag off, no AI endpoint is reachable.
- [ ] **[test]** Rejecting every draft element leaves the venue unchanged; an AI timeout shows a clear message.

10. Scope, defaults and what the human provides

Out of scope for now: user accounts and teams; real crowd physics (body collisions, crush dynamics); 3D; live data feeds from real turnstiles; true layout optimisation; mobile editing; public transport timetables. Avoid designs that would make accounts or real data feeds impossible later.

Parked ideas (not in this version): VIP, accessible and staff lanes with agent mixes; taxi, bus and drop-off kerbs with dwell times; weather (speed multiplier, bunched arrivals); staff reassignment when queues grow; reversible lanes and re-entry; one-way aisles; sensitivity (tornado) plots.

Decided defaults (use these without asking):

Question	Default
Product name	Bottleneck
Paid APIs	None in the core project. The AI draft is optional, Phase 8, off by default.
Logins	None. Unguessable venue IDs with a secret edit key (section 8).
Example venues	Belgrade Fair fair day, Belgrade Fair concert in a hall, stand-alone parking lot, drawn from OpenStreetMap with attribution, not Google Maps

Question	Default
Hosting	Free static hosting until Phase 4, then one small VPS
Units	Metres, seconds, km/h; 24-hour clock relative to event start
Real data	None. Section 5 defaults plus the synthetic log generator.
Run storage	Metrics, parameters and the time-aggregated heatmap, not every frame
Max agents	20,000 pedestrians plus 5,000 cars per run (soft limit with a warning)
Language	English

What the owner provides, and when:

- Phase 0: a GitHub repository and a free static-hosting account (Vercel, Cloudflare Pages or GitHub Pages) connected to it.
- Phase 1: a reference machine for the performance check (his own laptop is fine), and a look at the demo, since the agent cannot judge how a GPU-rendered canvas looks.
- Phase 4: a VPS with Docker installed (2 vCPU, 4 GB RAM is enough; about €5/month, or student credits such as those in the GitHub Student Developer Pack), SSH access for deployment, and a domain or subdomain pointed at it.
- Phase 8, only if chosen: an Anthropic API key with a spend limit set in the Anthropic console.

11. Working agreement for the coding agent

Work one phase at a time and leave the app runnable after every phase. The owner will study the code after it is built, so every phase also produces learning material.

1. **Start of a phase:** write a short plan in `docs/plans/phase-N.md`: what will be built, in what order, and the risks.
2. **During:** make engineering decisions yourself and record the significant ones (section 2). Write tests for engine rules, invariants and API endpoints as you go. Use the libraries named in section 8 unless there is a recorded reason not to.

3. **End of a phase:** tick off the acceptance checks in section 9. Run every **[test]** check in CI. List every **[human]** check in the phase summary for the owner, with exactly what to look at. Use Playwright screenshots where they help the owner check quickly.
4. **Phase summary:** what was built, what was deferred, which section 10 defaults were used, known limitations.
5. **Learning notes (Fixed):** one file per phase, `docs/how-it-works/phase-N.md`, in plain language for a junior developer: the design, the main data flow, why the key choices were made, and 5 interview-style questions with short answers.
6. **Never leave a phase half-wired.** If time runs out, cut scope inside the phase and say so; do not skip tests.
7. **Commits:** small, descriptive. One branch per phase, merged when its checks pass. Keep the live demo deployed from the main branch.

12. Sources

The prior-art tools named in section 1. The default parameters in section 5, including the turnstile, barrier and screening timings suggested in review, come from general knowledge and were not checked against published sources; they stay labelled "assumed". The Weidmann speed–density constants are the commonly cited textbook values.

- PTV Vissim: event traffic simulation for event organizers
- Oasys MassMotion: pedestrian simulation
- SimPlan: people-flow simulation services